QueueLess

A Virtual Queue System for University Services

Web Engineering

Semester Project Proposal

Project Team Size: 2 Students
Team Member 1: Shaheer Ahmed Siddiqui _ 053-24-0038
Team Member 2: Sumera Naz _ 053-24-0002
Course Instructor: Sir Nisar Ahmed
Department: Software Engineering Sec A
University: Sukkur IBA University
Submission Date: 24-09-2026

Contents

1. Project Title....4
2. Abstract....4
3. Introduction....4
4. Problem Statement....4
5. Proposed Solution....5
6. Project Objectives....5
7. Target Users....5
8. Project Scope....5
9. System Features....6
10. System Modules....6
    Authentication and User Management....6
    Service Management....6
    Virtual Queue Management....6
    Token Management....6
    Queue Tracking....7
    Estimated Waiting Time....7
    Staff Queue Management....7
    Counter Management....7
    Notifications....7
    Queue Cancellation....7
    Token Skip....7
    Queue History....7
    Admin Dashboard....7
    Analytics and Reports....7
11. User Roles....7
    Student....7
    Staff....8
    Administrator....8
12. Complete System Workflow....8
    Estimated Waiting Time....9
13. Functional Requirements....10
14. Non-Functional Requirements....11
15. Technology Stack....11
16. System Architecture....11
17. Database Design....12
18. Database Tables and Relationships....12

users 12
services 12
counters 13
queue_tokens 13
queue_history 13
Relationships Between Tables 13
19. REST API Overview 14
Authentication 14
Services 14
Queue 14
Admin 14
20. Frontend Pages 15
Public Pages 15
Student Pages 15
Staff Pages 15
Admin Pages 15
21. Security 15
22. Testing Plan 16
23. Expected Benefits 16
24. Project Innovation 16
25. Project Limitations 17
26. Future Enhancements 17
27. Development Plan 17
28. Team Responsibilities 18
Team Member 1 — Backend Developer 18
Team Member 2 — Frontend Developer 18
Shared Responsibilities 18
29. Expected Final Output 18
30. Conclusion 19

1. Project Title

QueueLess — A Virtual Queue Management System for University Services

2. Abstract

Students at most universities spend a lot of time standing in physical lines to get simple services done, such as paying fees, getting documents verified, or visiting the admissions office. QueueLess is a proposed web application that moves this waiting process online. Instead of standing in a line, a student joins a virtual queue through the website and gets a token. The student can then check their position, see the estimated waiting time, and come back to the office only when their turn is close. This proposal explains the idea, the features, the database design, the technology stack, and the development plan for building QueueLess as a Web Engineering semester project.

3. Introduction

University service offices such as the Fee Office, Student Affairs, or the Examination Office usually serve students on a first-come-first-served basis using paper tokens or simple physical lines. Students often do not know how long they will have to wait, so they end up standing in the office for a long time, even if their actual service only takes a few minutes.

QueueLess is our proposed solution to this problem. It is a web-based system that lets students join a queue online, see their position and estimated wait time in real time (through page refresh), and only go to the office when it is close to their turn. Staff members get a dashboard to manage the queue, and administrators get tools to manage services, counters, and staff, and to view reports about how the system is being used.

4. Problem Statement

In a traditional queue system, students do not know their exact position or how long they will wait. This creates the following problems:

- Students waste time physically standing in line instead of using that time productively.
- Office staff cannot manage the queue efficiently, especially during busy hours.
- There is no record of how long each service takes, so the university cannot measure or improve its own service time.
- Multiple students often crowd around one counter, which is uncomfortable and disorganized.
- There is no easy way for administrators to see which services are slow or under pressure.

QueueLess is designed to reduce these problems by moving the waiting process online and giving staff and admins better visibility and control.

5. Proposed Solution

QueueLess allows a student to join a queue for any university service from the website, instead of physically going and taking a paper token. Once the student joins, the system generates a unique virtual token for that service. The student can then see their token number, the token currently being served, the number of people ahead of them, and an estimated waiting time.

The student does not need to stand in the office during this waiting time. They can check the queue status on the website whenever they want. When their turn is close, the system shows a notification on the dashboard asking them to come to the counter. On the other side, staff members get a simple dashboard where they can call the next token, skip a token if the student is not present, and mark a token as completed once the service is done. Administrators can manage services, staff, and counters, and can view statistics about waiting time and service time.

6. Project Objectives

- To design and build a web-based virtual queue system for university service offices.
- To reduce the physical waiting time students spend standing in lines.
- To give students clear, real-time information about their queue position and estimated waiting time.
- To provide staff members with a simple dashboard to manage their queue.
- To give administrators tools to manage services, staff, counters, and view reports.
- To apply Web Engineering concepts such as REST APIs, relational database design, authentication, and responsive frontend design in one complete project.

7. Target Users

QueueLess is designed for three types of users inside a university:

User	Description
Students	Use the system to join queues for services, track their token, and view their queue history.
Staff	Work at university service offices and use the system to manage and serve the queue.
Administrators	Manage the overall system, including services, staff, counters, and reports.

8. Project Scope

The semester version of QueueLess focuses on university service offices such as Student Affairs, the Fee Office, Admissions, the Examination Office, IT Support, and Document Verification. The scope of the project for this semester includes:

- User authentication and role-based access for students, staff, and administrators

- Service listing and management
- Virtual queue joining and token generation
- Real-time (page-refresh based) queue tracking
- Estimated waiting time calculation
- Turn notifications on the dashboard
- Staff queue management (call next, skip, complete)
- Admin management of users, services, staff, and counters
- Queue history and basic analytics
- A responsive web interface that works on desktop and mobile browsers

The project is kept realistic in size for a two-person team to design, build, and test within one semester.

9. System Features

From a student's point of view, once they join a queue, they can see the following information on their dashboard:

- Their own token number
- The token currently being served
- The number of people ahead of them
- The estimated waiting time
- The counter assigned to serve them
- The current status of their token (waiting, called, serving, completed, cancelled, or skipped)

This information updates whenever the student refreshes or revisits the queue status page, so they always know where they stand without needing to be physically present in the office.

10. System Modules

Authentication and User Management

Handles registration, login, and role-based access for students, staff, and administrators. Passwords are stored securely, and each user is given a role that decides what they can access in the system.

Service Management

Lets administrators create, update, open, or close university services such as the Fee Office or Admissions. Each service has an average service time used for waiting time calculation.

Virtual Queue Management

Manages the queue for each service, including adding students to the queue and keeping track of their order.

Token Management

Generates a unique token for each student when they join a queue and manages the token's state through its lifecycle, from waiting to completed.

Queue Tracking

Let's students check their token number, people ahead, and the currently serving token at any time.

Estimated Waiting Time

Calculates how long a student is likely to wait, based on their position in the queue and the average service time of that service.

Staff Queue Management

Gives staff a dashboard to see waiting students, call the next token, skip a token, and mark a token as completed.

Counter Management

Lets administrators create counters for each service and assign staff members to them.

Notifications

Shows an on-screen notification to the student when their turn is approaching, based on their position in the queue.

Queue Cancellation

Allows a student to cancel their own token if they no longer need the service.

Token Skip

Allows staff to skip a token if the student does not show up when called, so the queue can keep moving.

Queue History

Keeps a record of every completed, cancelled, or skipped token for both students and staff to review later.

Admin Dashboard

Gives administrators a central place to manage users, staff, services, and counters.

Analytics and Reports

Shows waiting-time and service-time statistics so the admin can see how each service is performing.

11. User Roles

Student

- Register and login
- View available services

- Join a queue
- Receive a virtual token
- View queue position
- View estimated waiting time
- Monitor the current serving token
- Cancel their token
- Receive turn notifications
- View previous queue history

Staff

- Login
- View their assigned service queue
- See waiting students
- Call the next token
- Skip a token
- Complete a service
- View service history

Administrator

- Manage users
- Manage staff
- Create and update services
- Open or close services
- Manage counters
- Assign staff to counters
- View queue history
- View waiting-time and service-time statistics
- Monitor overall system activity

12. Complete System Workflow

The table below shows the token lifecycle used in QueueLess:

From State	To State	Meaning
WAITING	CALLED	Staff has called this token; the student is being asked to come to the counter.
CALLED	SERVING	Staff has started serving this student at the counter.

From State	To State	Meaning
SERVING	COMPLETED	The service has finished for this student.
WAITING	CANCELLED	The student cancelled the token before it was called.
WAITING	SKIPPED	The student did not respond when called, so staff skipped the token.

The full queue process, step by step, is as follows:

1. Student logs into QueueLess.
2. Student views the list of available university services.
3. Student selects the service they need.
4. Student clicks "Join Queue" for that service.
5. The backend checks whether the service is currently open and whether the student is already in a queue for it.
6. The system generates a unique token for the student.
7. The student sees their token number, the current token being served, the number of people ahead, and the estimated waiting time.
8. The student can leave the physical waiting area and check the queue status online whenever they want.
9. Staff calls the next token from their staff dashboard.
10. The student receives an on-screen notification when their turn is approaching.
11. The student arrives at the assigned counter.
12. Staff provides the requested service.
13. Staff marks the token as completed once the service is finished.
14. The system records the waiting time and service time for that token.
15. The completed transaction is saved in the queue history and used for analytics.

Estimated Waiting Time

The system calculates the estimated waiting time for a student using this simple formula:

Estimated Waiting Time = Number of People Ahead × Average Service Time

For example, if there are 5 people ahead of a student and the average service time for that service is 5 minutes, the estimated waiting time would be:

5 × 5 minutes = 25 minutes

The average service time for each service can later be recalculated from recent completed transactions in the queue_history table, so the estimate becomes more accurate over time instead of staying fixed.

13. Functional Requirements

S. No	Requirement	Description
FR-01	Registration	The system shall allow a new student to register with a name, email, and password.
FR-01	Login	The system shall allow registered users to log in using their email and password.
FR-01	Role-based access	The system shall give each user access only to the pages and actions allowed for their role (student, staff, or admin).
FR-01	Service viewing	The system shall let students view the list of available services and their status (open or closed).
FR-01	Joining queues	The system shall let a student join the queue for an open service.
FR-01	Token generation	The system shall generate a unique token for every student who joins a queue.
FR-01	Queue tracking	The system shall show the student their token number, people ahead, and the currently serving token.
FR-01	Estimated waiting time	The system shall calculate and display the estimated waiting time for each student in the queue.
FR-01	Queue cancellation	The system shall let a student cancel their own token while it is still waiting.
FR-01	Staff queue management	The system shall let staff view the list of students waiting in their assigned queue.
FR-01	Token calling	The system shall let staff call the next token in the queue.
FR-01	Token skipping	The system shall let staff skip a token if the student does not respond.
FR-01	Service completion	The system shall let staff mark a token as completed after the service is provided.
FR-01	Service management	The system shall let administrators create, update, open, and close services.
FR-01	Counter management	The system shall let administrators create counters and assign staff to them.
FR-01	Queue history	The system shall keep a record of completed, cancelled, and skipped tokens.
FR-01	Analytics	The system shall show waiting-time and service-time statistics to the administrator.
FR-01	Notifications	The system shall show an on-screen notification to a student when their turn is approaching.

14. Non-Functional Requirements

S. No	Requirement	Description
NFR-01	Security	User passwords must be stored in hashed form, and access to APIs must be controlled based on user role.
NFR-02	Performance	Queue status pages should load quickly and reflect up-to-date token information.
NFR-03	Usability	The interface should be simple and easy to understand for students, staff, and admins who are not technical.
NFR-04	Reliability	The system should correctly track token states without losing or duplicating tokens.
NFR-05	Maintainability	The code should be organized into clear modules so it is easy to update or extend later.
NFR-06	Scalability	The database and backend structure should be able to support additional services or counters without major redesign.
NFR-07	Responsive design	The website should work well on both desktop and mobile browsers.

15. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Bootstrap, JavaScript
Backend	Node.js, Express.js
Database	MySQL
Version Control	Git and GitHub
API Testing	Postman
Code Editor	VS Code
Database Design Tool	MySQL Workbench

This stack was chosen because it is well suited for a Web Engineering course project: it covers frontend development, backend REST API development, and relational database design, while staying realistic for a two-person team to learn and use within one semester.

16. System Architecture

QueueLess follows a standard three-layer web application architecture. The frontend (built with HTML, CSS, Bootstrap, and JavaScript) is responsible for what the user sees and interacts with. It sends requests to the backend using REST APIs. The backend (built with Node.js and Express.js) contains the application logic, such as generating tokens, checking queue rules, and calculating waiting time. The backend communicates with the MySQL database to store and retrieve data such as users, services, and queue tokens.

The basic flow of a request is as follows:

1. The user interacts with a page in the browser (frontend).
2. The frontend sends a request to a REST API endpoint on the backend.
3. The backend (Express.js) processes the request and applies business logic.
4. The backend reads from or writes to the MySQL database.
5. The backend sends a response back to the frontend.
6. The frontend updates the page to show the result to the user.

17. Database Design

QueueLess uses a MySQL relational database to store users, services, counters, and queue tokens. The database is designed around five main tables, described below.

18. Database Tables and Relationships

users

Column	Description
id	Unique identifier for the user
name	Full name of the user
email	Email address, used for login
password	Hashed password
role	Role of the user: student, staff, or admin
created_at	Date and time the account was created

services

Column	Description
id	Unique identifier for the service
name	Name of the service, e.g. Fee Office
description	Short description of the service
average_service_time	Average time (in minutes) taken to serve one student
status	Whether the service is open or closed
created_at	Date and time the service was created

counters

Column	Description
id	Unique identifier for the counter
service_id	The service this counter belongs to
staff_id	The staff member assigned to this counter
counter_number	Display number of the counter, e.g. Counter 1
status	Whether the counter is active or inactive

queue_tokens

Column	Description
id	Unique identifier for the token
service_id	The service this token belongs to
user_id	The student who owns this token
token_number	The token number given to the student
status	Current state of the token (waiting, called, serving, completed, cancelled, skipped)
joined_at	Time the student joined the queue
called_at	Time the token was called by staff
completed_at	Time the service was completed

queue_history

Column	Description
id	Unique identifier for the history record
token_id	The token this history record belongs to
waiting_time	Total time the student waited, in minutes
service_time	Total time taken to serve the student, in minutes
served_at	Date and time the service was completed

Relationships Between Tables

- A user (student) can have many queue_tokens, but each queue_token belongs to only one user.
- A service can have many queue_tokens, but each queue_token belongs to only one service.
- A service can have many counters, but each counter belongs to only one service.
- A user with the staff role can be assigned to one counter through the staff_id field in the counters table.
- Each queue_token can have one related record in queue_history, created once the token reaches a final state such as completed, cancelled, or skipped.

19. REST API Overview

The backend exposes REST APIs for the frontend to use. A high-level overview of the main endpoints is given below. The exact routes, request bodies, and response formats may be adjusted during implementation.

Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register a new student account
POST	/api/auth/login	Log in and receive an authentication token

Services

Method	Endpoint	Description
GET	/api/services	Get the list of all services
GET	/api/services/:id	Get details of a single service
POST	/api/services	Create a new service (admin only)
PUT	/api/services/:id	Update an existing service (admin only)
DELETE	/api/services/:id	Delete a service (admin only)

Queue

Method	Endpoint	Description
POST	/api/queues/join	Join the queue for a service and get a token
GET	/api/queues/status	Get the current queue status for a token
POST	/api/queues/cancel	Cancel a waiting token
POST	/api/queues/call-next	Call the next token (staff only)
POST	/api/queues/complete	Mark a token as completed (staff only)
POST	/api/queues/skip	Skip a token (staff only)

Admin

Method	Endpoint	Description
GET	/api/admin/users	Get the list of all users
GET	/api/admin/statistics	Get waiting-time and service-time statistics
GET	/api/admin/history	Get the full queue history

20. Frontend Pages

Public Pages

- Home
- Login
- Register
- Services

Student Pages

- Dashboard
- Join Queue
- Queue Status
- History
- Profile

Staff Pages

- Dashboard
- Queue Management
- History
- Profile

Admin Pages

- Dashboard
- Users
- Services
- Counters
- Staff
- History
- Analytics

21. Security

QueueLess will follow standard web security practices for a project of this scale:

- Passwords will be hashed using bcrypt before being stored in the database, so plain-text passwords are never saved.
- JWT (JSON Web Token) based authentication will be used to confirm a user's identity on each request.

- Role-based authorization will make sure a student cannot access staff or admin-only APIs, and staff cannot access admin-only APIs.
- Input validation will be applied on both frontend and backend to prevent invalid or harmful data from being submitted.
- Sensitive API routes will be protected and will require a valid token before allowing access.
- These measures together help prevent unauthorized users from viewing or changing data that does not belong to them.

22. Testing Plan

Testing will be done throughout development, not just at the end. The following types of testing are planned:

Type	Description
Functional testing	Checking that each feature (joining a queue, calling a token, etc.) works as expected.
API testing with Postman	Testing each REST API endpoint separately for correct responses and error handling.
Authentication testing	Checking that login and token-based authentication work correctly.
Role/authorization testing	Checking that students, staff, and admins can only access what their role allows.
Input validation testing	Checking that invalid or missing input is rejected with a proper error message.
Queue logic testing	Checking that token generation, calling, skipping, and completing follow the correct order and rules.
UI testing	Checking that all pages display correctly and buttons/forms work as intended.
Responsive testing	Checking that the website looks and works correctly on different screen sizes.

23. Expected Benefits

- Students save time by not having to stand physically in line.
- Staff can manage their queue more easily through a simple dashboard.
- Administrators get visibility into how each service is performing through waiting-time and service-time data.
- University offices become more organized, with fewer students crowding around counters.
- The recorded history and analytics can help the university identify which services need more staff or resources.

24. Project Innovation

While virtual queue systems already exist in some real-world settings such as banks and hospitals, QueueLess applies this idea specifically to university service offices, which usually still rely on paper tokens or informal lines. The project combines queue management, real-time-style tracking, and basic analytics in one system designed around the specific needs of student services such as Fee Offices, Admissions, and Examination Offices. This

makes it a practical and relevant application of Web Engineering concepts, rather than just a theoretical exercise.

25. Project Limitations

To keep the project realistic for a two-person team within one semester, the following will not be included in this version:

- Native mobile applications
- GPS tracking
- SMS gateway integration
- Complex AI-based prediction
- Direct integration with university ERP systems
- Payment processing

26. Future Enhancements

The following features are not part of the semester project but could be added in the future if the project is continued:

- SMS notifications
- Email notifications
- A dedicated mobile application
- QR code check-in at counters
- More advanced queue prediction based on historical data
- WebSocket-based real-time updates instead of page refresh
- Support for multiple university branches
- Integration with the university's ERP system

27. Development Plan

The project will be developed in the following phases over the semester:

Phase	Description
Phase 1	Requirements gathering and planning
Phase 2	Database and system design
Phase 3	Backend setup and authentication
Phase 4	Queue and service REST APIs
Phase 5	Student frontend pages

Phase	Description
Phase 6	Staff dashboard
Phase 7	Admin dashboard
Phase 8	Waiting time calculation and notifications
Phase 9	Testing and bug fixing
Phase 10	Documentation, deployment, and final presentation

28. Team Responsibilities

Team Member 1 — Backend Developer

- Node.js and Express.js development
- Designing and building REST APIs
- MySQL database implementation
- Authentication and authorization logic
- Queue and token logic

Team Member 2 — Frontend Developer

- HTML, CSS, and Bootstrap based page design
- JavaScript for dynamic behavior
- Building student, staff, and admin dashboards
- Connecting frontend pages to backend APIs

Shared Responsibilities

- Requirements analysis
- Database design
- Testing
- Git and GitHub version control
- Documentation
- Final presentation

29. Expected Final Output

By the end of the semester, the project is expected to deliver:

- A working web application with separate views for students, staff, and administrators
- A functioning REST API backend connected to a MySQL database
- The ability to join a queue, receive a token, track queue status, and receive turn notifications

- Staff tools to call, skip, and complete tokens
- Admin tools to manage services, staff, counters, and view basic analytics
- Project documentation and a final presentation demonstrating the system

30. Conclusion

QueueLess addresses a problem that most students face regularly: spending too much time physically waiting in university service offices. By moving the queue online, the system gives students control over their own time while giving staff and administrators better tools to manage service delivery.

The project uses a technology stack (HTML, CSS, Bootstrap, JavaScript, Node.js, Express.js, and MySQL) that is well matched to a Web Engineering course, since it covers frontend design, backend REST API development, and relational database design in one connected system. The scope has been kept realistic for a two-person team to design, build, and test within one semester, while still covering the core concepts expected in a Web Engineering project. For these reasons, QueueLess is both a useful, practical idea and a realistic and achievable semester project.